

Додаток 1. Лістинг коду програми

apps/lab1.cpp

```
1 #include <FileSource.hpp>
2 #include <Log.hpp>
3 #include <SymbolStore.hpp>
4 #include <Table.hpp>
5 #include <Tokenizer.hpp>
6 #include <iostream>
7
8 std::string getTypeLabel(SymbolType type) {
9     switch (type) {
10         case SymbolType::Delimiter:
11             return "Delimiter";
12         case SymbolType::Keyword:
13             return "Keyword";
14         case SymbolType::Literal:
15             return "Literal";
16         case SymbolType::Identifier:
17             return "Identifier";
18     }
19 }
20
21 int main(int argc, char* argv[]) {
22     if (argc < 2) {
23         std::cerr << "Usage: lab1 <filename>" << std::endl;
24         return 1;
25     }
26
27     FileSource source(argv[1]);
28     SymbolStore symbols;
29     CharacterAttributes attributes;
30     Tokenizer tokenizer(source, symbols, attributes);
31     tokenizer.scan();
32
33     Table("\nTokens", tokenizer.tokens())
34         .addColumn(
35             "Code", "{:>5}",
36             [](const auto& t) { return std::to_string(t.code); }
37         )
38         .addColumn(
39             "Row", "{:>3}",
40             [](const auto& t) { return std::to_string(t.row + 1); }
41         )
42         .addColumn(
43             "Col", "{:>3}",
44             [](const auto& t) { return std::to_string(t.column + 1); }
45         )
46         .addColumn(
47             "Type", "{:<10}",
48             [&](const auto& t) {
49                 return getTypeLabel(symbols.lookupType(t.code));
50             }
51         )
52         .addColumn(
53             "Value", "{:<30}",
54             [&](const auto& t) { return std::string(symbols.lookup(t.code)); }
55         )
56         .print();
57
58     Log("Tokenizer", tokenizer.errors())
59         .setFormatter([](const auto& err) {
60             return std::format(
```

```

61         "Error [{}:{}]: {}", err.row + 1, err.column + 1, err.message
62     );
63 })
64 .print();
65
66 Table("\nIdentifiers", symbols.identifiers())
67     .addColumn(
68         "Code", "{:>5}",
69         [](const auto& pair) { return std::to_string(pair.second); }
70     )
71     .addColumn(
72         "Value", "{:<30}", [](const auto& pair) { return pair.first; }
73     )
74     .print();
75
76 Table("\nLiterals", symbols.literals())
77     .addColumn(
78         "Code", "{:>5}",
79         [](const auto& pair) { return std::to_string(pair.second); }
80     )
81     .addColumn(
82         "Value", "{:<30}", [](const auto& pair) { return pair.first; }
83     )
84     .print();
85
86 return 0;
87 }

```

src/CharacterAttributes/CharacterAttributes.hpp

```
1 #pragma once
2
3 #include <array>
4
5 #include "Constants.hpp"
6
7 enum class Attribute {
8     Whitespace,
9     Digit,
10    Letter,
11    Delimiter,
12    Comment,
13    Invalid,
14 };
15
16 class CharacterAttributes {
17 private:
18     std::array<Attribute, ASCII_TABLE_SIZE> attributes;
19
20 public:
21     CharacterAttributes();
22
23     [[nodiscard]] Attribute lookup(char character) const;
24 };
```

src/CharacterAttributes/CharacterAttributes.cpp

```
1 #include "CharacterAttributes.hpp"
2
3 CharacterAttributes::CharacterAttributes() {
4     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
5         attributes[i] = Attribute::Invalid;
6     }
7
8     // Whitespace
9     attributes[' '] = Attribute::Whitespace;
10    attributes['\t'] = Attribute::Whitespace;
11    attributes['\n'] = Attribute::Whitespace;
12    attributes['\v'] = Attribute::Whitespace;
13    attributes['\f'] = Attribute::Whitespace;
14    attributes['\r'] = Attribute::Whitespace;
15
16    // Digits
17    for (char ch = '0'; ch <= '9'; ch++) {
18        attributes[ch] = Attribute::Digit;
19    }
20
21    // Letters (A-Z)
22    for (char ch = 'A'; ch <= 'Z'; ch++) {
23        attributes[ch] = Attribute::Letter;
24    }
25
26    // Delimiters
27    attributes[';'] = Attribute::Delimiter;
28    attributes['.'] = Attribute::Delimiter;
29    attributes['='] = Attribute::Delimiter;
30    attributes['\''] = Attribute::Delimiter;
31    attributes[','] = Attribute::Delimiter;
32    attributes[')'] = Attribute::Delimiter;
33    attributes['$'] = Attribute::Delimiter;
34
35    // Comment start
36    attributes['('] = Attribute::Comment;
37 }
38
39 Attribute CharacterAttributes::lookup(char character) const {
40     return attributes[static_cast<unsigned char>(character)];
41 }
```

src/Constants/Constants.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 constexpr size_t ASCII_TABLE_SIZE = 256;
```

src/Log/Log.hpp

```
1 #pragma once
2
3 #include <functional>
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 template <typename T>
9 class Log {
10 private:
11     std::string name;
12     std::vector<T> data;
13     std::function<std::string(const T&)> formatter;
14     std::ostream& out;
15
16 public:
17     Log(std::string name, std::vector<T> data, std::ostream& out = std::cout);
18
19     Log& setFormatter(std::function<std::string(const T&)> formatter);
20
21     void print() const;
22 };
23
24 #include "Log.tpp"
```

src/Log/Log.hpp

```
1 template <typename T>
2 Log<T>::Log(std::string name, std::vector<T> data, std::ostream& out) :
3     name(std::move(name)), data(std::move(data)), out(out) {}
4
5 template <typename T>
6 Log<T>& Log<T>::setFormatter(std::function<std::string(const T&)> formatter) {
7     this->formatter = std::move(formatter);
8
9     return *this;
10 }
11
12 template <typename T>
13 void Log<T>::print() const {
14     for (const auto& item : data) {
15         out << name << " | " << formatter(item) << "\n";
16     }
17 }
```

src/Source/Source.hpp

```
1 #pragma once
2
3 #include <SourcePosition.hpp>
4
5 class Source {
6 protected:
7     SourcePosition position;
8
9 public:
10     Source();
11     Source(SourcePosition position);
12
13     virtual ~Source() = default;
14
15     virtual char current() = 0;
16     virtual char read() = 0;
17     virtual bool done() = 0;
18
19     [[nodiscard]] unsigned int row() const;
20     [[nodiscard]] unsigned int column() const;
21 };
```


src/Source/Source.cpp

```
1 #include "Source.hpp"
2
3 Source::Source() = default;
4
5 Source::Source(SourcePosition position) : position(position) {}
6
7 unsigned int Source::row() const {
8     return position.row();
9 }
10
11 unsigned int Source::column() const {
12     return position.column();
13 }
```

src/Source/FileSource/FileSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <fstream>
5 #include "Source.hpp"
6
7 class FileSource : public Source {
8 private:
9     std::ifstream file;
10     char character;
11
12 public:
13     FileSource(const std::string& filePath);
14
15     char current() override;
16     char read() override;
17     bool done() override;
18 };
```

src/Source/FileSource/FileSource.cpp

```
1 #include "FileSource.hpp"
2
3 FileSource::FileSource(const std::string& filePath) :
4     file(filePath),
5     character(static_cast<char>(file.get())) {}
6
7 char FileSource::current() {
8     return character;
9 }
10
11 char FileSource::read() {
12     position.advance(character);
13     character = static_cast<char>(file.get());
14
15     return character;
16 }
17
18 bool FileSource::done() {
19     return file.eof();
20 }
```

src/Source/StringSource/StringSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include "Source.hpp"
5
6 class StringSource : public Source {
7 private:
8     std::string source;
9     std::string::iterator iterator;
10
11 public:
12     StringSource(std::string source);
13
14     char current() override;
15     char read() override;
16     bool done() override;
17 };
```

src/Source/StringSource/StringSource.cpp

```
1 #include "StringSource.hpp"
2
3 StringSource::StringSource(std::string source) :
4     source(std::move(source)),
5     iterator(this->source.begin()) {}
6
7 char StringSource::current() {
8     return *iterator;
9 }
10
11 char StringSource::read() {
12     position.advance(*iterator);
13     iterator++;
14
15     return *iterator;
16 }
17
18 bool StringSource::done() {
19     return iterator == source.end();
20 }
```

src/SourcePosition/SourcePosition.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 class SourcePosition {
6 private:
7     uint8_t tabSize = 4;
8
9     unsigned int _row = 0;
10    unsigned int _column = 0;
11
12 public:
13     SourcePosition();
14     SourcePosition(uint8_t tabSize);
15
16     void advance(char character);
17     [[nodiscard]] unsigned int row() const;
18     [[nodiscard]] unsigned int column() const;
19 };
```

src/SourcePosition/SourcePosition.cpp

```
1 #include "SourcePosition.hpp"
2
3 SourcePosition::SourcePosition() : SourcePosition(4) {}
4
5 SourcePosition::SourcePosition(uint8_t tabSize) : tabSize(tabSize) {};
6
7 void SourcePosition::advance(char character) {
8     switch (character) {
9         case '\n':
10             _row++;
11             _column = 0;
12             break;
13
14         case '\t':
15             _column += tabSize;
16             break;
17
18         default:
19             _column++;
20             break;
21     }
22 }
23
24 unsigned int SourcePosition::row() const {
25     return _row;
26 }
27
28 unsigned int SourcePosition::column() const {
29     return _column;
30 }
```

src/SymbolStore/SymbolStore.hpp

```
1  #pragma once
2
3  #include <array>
4  #include <string>
5  #include <unordered_map>
6  #include <vector>
7
8  #include "Constants.hpp"
9
10 enum class SymbolType {
11     Delimiter,
12     Keyword,
13     Literal,
14     Identifier,
15 };
16
17 const size_t DELIMITERS_OFFSET = 0;
18 const size_t KEYWORDS_OFFSET = ASCII_TABLE_SIZE;
19 const size_t LITERALS_OFFSET = 500;
20 const size_t IDENTIFIERS_OFFSET = 1000;
21 const size_t MAX_TOKENS = 2000;
22
23 class SymbolStore {
24 private:
25     std::array<std::string, MAX_TOKENS> symbols;
26     std::unordered_map<std::string, size_t> _keywords;
27     std::unordered_map<std::string, size_t> _identifiers;
28     std::unordered_map<std::string, size_t> _literals;
29
30     void declareDelimiter(char character);
31     void declareKeyword(const std::string& keyword);
32
33 public:
34     SymbolStore();
35
36     size_t resolveKeyword(const std::string& keyword);
37     size_t resolveIdentifier(const std::string& identifier);
38     size_t resolveLiteral(const std::string& literal);
39
40     [[nodiscard]] bool isKeyword(const std::string& token) const;
41     [[nodiscard]] const std::string& lookup(size_t code) const;
42     [[nodiscard]] SymbolType lookupType(size_t code) const;
43     [[nodiscard]] std::vector<std::pair<std::string, size_t>>
44     identifiers() const;
45     [[nodiscard]] std::vector<std::pair<std::string, size_t>> literals() const;
46 };
```


src/SymbolStore/SymbolStore.cpp

```
1 #include "SymbolStore.hpp"
2
3 #include <algorithm>
4 #include <format>
5 #include <stdexcept>
6
7 SymbolStore::SymbolStore() {
8     // Initialize ASCII symbols
9     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
10         declareDelimiter(static_cast<char>(i));
11     }
12
13     // Initialize keywords
14     declareKeyword("PROGRAM");
15     declareKeyword("CONST");
16     declareKeyword("BEGIN");
17     declareKeyword("END");
18     declareKeyword("EXP");
19 }
20
21 void SymbolStore::declareDelimiter(const char character) {
22     symbols[static_cast<unsigned char>(character)] = std::string(1, character);
23 }
24
25 void SymbolStore::declareKeyword(const std::string& keyword) {
26     if (_keywords.contains(keyword)) {
27         throw std::invalid_argument(
28             std::format("Keyword '{}{}' is already declared", keyword)
29         );
30     }
31
32     size_t code = KEYWORDS_OFFSET + _keywords.size();
33
34     if (code >= LITERALS_OFFSET) {
35         throw std::overflow_error("Exceeded maximum number of keywords");
36     }
37
38     _keywords[keyword] = code;
39     symbols[code] = keyword;
40 }
41
42 size_t SymbolStore::resolveKeyword(const std::string& keyword) {
43     if (!_keywords.contains(keyword)) {
44         throw std::invalid_argument(
45             std::format("{}{} is not a keyword", keyword)
46         );
47     }
48
49     return _keywords[keyword];
50 }
51
52 size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
53     if (!_identifiers.contains(identifier)) {
54         size_t code = IDENTIFIERS_OFFSET + _identifiers.size();
55
56         if (code >= MAX_TOKENS) {
57             throw std::overflow_error("Exceeded maximum number of identifiers");
58         }
59
60         _identifiers[identifier] = code;
61         symbols[code] = identifier;
62     }
```

```

63
64     return _identifiers[identifier];
65 }
66
67 size_t SymbolStore::resolveLiteral(const std::string& literal) {
68     if (!_literals.contains(literal)) {
69         size_t code = LITERALS_OFFSET + _literals.size();
70
71         if (code >= IDENTIFIERS_OFFSET) {
72             throw std::overflow_error("Exceeded maximum number of literals");
73         }
74
75         _literals[literal] = code;
76         symbols[code] = literal;
77     }
78
79     return _literals[literal];
80 }
81
82 bool SymbolStore::isKeyword(const std::string& token) const {
83     return _keywords.contains(token);
84 }
85
86 SymbolType SymbolStore::lookupType(size_t code) const {
87     if (code < KEYWORDS_OFFSET) {
88         return SymbolType::Delimiter;
89     }
90
91     if (code < LITERALS_OFFSET) {
92         return SymbolType::Keyword;
93     }
94
95     if (code < IDENTIFIERS_OFFSET) {
96         return SymbolType::Literal;
97     }
98
99     if (code < MAX_TOKENS) {
100         return SymbolType::Identifier;
101     }
102
103     throw std::out_of_range(
104         std::format("Symbol code {} is out of range", code)
105     );
106 }
107
108 const std::string& SymbolStore::lookup(size_t code) const {
109     if (code >= symbols.size()) {
110         throw std::out_of_range(
111             std::format("Symbol code {} is out of range", code)
112         );
113     }
114
115     return symbols[code];
116 }
117
118 std::vector<std::pair<std::string, size_t>> SymbolStore::identifiers() const {
119     std::vector<std::pair<std::string, size_t>> result;
120
121     for (const auto& pair : _identifiers) {
122         result.emplace_back(pair.first, pair.second);
123     }
124
125     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
126         return a.second < b.second;
127     });
128
129     return result;

```

```
130 }
131
132 std::vector<std::pair<std::string, size_t>> SymbolStore::literals() const {
133     std::vector<std::pair<std::string, size_t>> result;
134
135     for (const auto& pair : _literals) {
136         result.emplace_back(pair.first, pair.second);
137     }
138
139     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
140         return a.second < b.second;
141     });
142
143     return result;
144 }
```

src/Table/Table.hpp

```
1  #pragma once
2
3  #include <format>
4  #include <functional>
5  #include <iostream>
6  #include <string>
7  #include <vector>
8
9  template <typename T>
10 class Table {
11 private:
12     struct Column {
13         std::string name;
14         std::string format;
15         std::function<std::string(const T&)> extractor;
16     };
17
18     std::string name;
19     std::vector<T> data;
20     std::vector<Column> columns;
21     std::ostream& out;
22
23     [[nodiscard]] std::string border() const;
24     [[nodiscard]] std::string header() const;
25     [[nodiscard]] std::string row(const T& item) const;
26     [[nodiscard]] size_t columnWidth(const Column& col) const;
27     static size_t parseWidth(const std::string& fmt);
28
29 public:
30     Table(std::string name, std::vector<T> data, std::ostream& out = std::cout);
31
32     Table& addColumn(
33         const std::string& colName,
34         const std::string& format,
35         std::function<std::string(const T&)> extractor
36     );
37
38     void print() const;
39 };
40
41 #include "Table.hpp"
```

```
1  template <typename T>
2  Table<T>::Table(std::string name, std::vector<T> data, std::ostream& out) :
3      name(std::move(name)), data(std::move(data)), out(out) {}
4
5  template <typename T>
6  Table<T>& Table<T>::addColumn(
7      const std::string& colName,
8      const std::string& format,
9      std::function<std::string(const T&)> extractor
10 ) {
11     columns.push_back({colName, format, std::move(extractor)});
12
13     return *this;
14 }
15
16 template <typename T>
17 void Table<T>::print() const {
18     if (!name.empty()) {
19         out << name << "\n";
20     }
21
22     std::string borderLine = border();
23
24     out << borderLine << "\n";
25     out << header() << "\n";
26     out << borderLine << "\n";
27
28     for (const auto& item : data) {
29         out << row(item) << "\n";
30     }
31
32     out << borderLine << "\n";
33 }
34
35 template <typename T>
36 std::string Table<T>::border() const {
37     std::string result = "+";
38
39     for (const auto& col : columns) {
40         size_t width = columnWidth(col);
41         result += std::string(width + 2, '-') + "+";
42     }
43
44     return result;
45 }
46
47 template <typename T>
48 std::string Table<T>::header() const {
49     std::string result = "|";
50
51     for (const auto& column : columns) {
52         result += std::format(" {:<{}} |", column.name, columnWidth(column));
53     }
54
55     return result;
56 }
57
58 template <typename T>
59 std::string Table<T>::row(const T& item) const {
60     std::string result = "|";
61
62     for (const auto& column : columns) {
```

```

63         std::string value = column.extractor(item);
64         result += " " +
65             std::vformat(column.format, std::make_format_args(value)) +
66             " |";
67     }
68
69     return result;
70 }
71
72 template <typename T>
73 size_t Table<T>::columnWidth(const Column& column) const {
74     return std::max(parseWidth(column.format), column.name.length());
75 }
76
77 template <typename T>
78 size_t Table<T>::parseWidth(const std::string& fmt) {
79     size_t width = 0;
80
81     for (char chr : fmt) {
82         if (chr >= '0' && chr <= '9') {
83             width = (width * 10) + (chr - '0');
84         }
85     }
86
87     return width;
88 }

```

src/Tokenizer/Tokenizer.hpp

```
1  #pragma once
2
3  #include <unordered_map>
4  #include <vector>
5
6  #include "CharacterAttributes.hpp"
7  #include "Source.hpp"
8  #include "SymbolStore.hpp"
9
10 struct Token {
11     size_t code;
12     size_t row;
13     size_t column;
14 };
15
16 struct Error {
17     std::string message;
18     size_t row;
19     size_t column;
20 };
21
22 class Tokenizer {
23 private:
24     Source& source;
25     SymbolStore& symbols;
26     CharacterAttributes& attributes;
27
28     char character;
29     std::string token;
30     size_t code;
31
32     std::vector<Token> _tokens;
33     std::vector<Error> _errors;
34
35     void scanWhitespaces();
36     void scanInteger();
37     void scanString();
38     void scanComment();
39     void scanDelimiter();
40     void scanInvalid();
41
42 public:
43     Tokenizer(
44         Source& source,
45         SymbolStore& symbols,
46         CharacterAttributes& attributes
47     );
48
49     void scan();
50
51     void addToken(const Token& token);
52     void addError(const Error& error);
53
54     [[nodiscard]] const std::vector<Token>& tokens() const;
55     [[nodiscard]] const std::vector<Error>& errors() const;
56 };
```

src/Tokenizer/Tokenizer.cpp

```
1  #include "Tokenizer.hpp"
2
3  #include <format>
4
5  #include "CharacterAttributes.hpp"
6
7  // Automata state: START
8  Tokenizer::Tokenizer(
9      Source& source,
10     SymbolStore& symbols,
11     CharacterAttributes& attributes
12 ) :
13     source(source),
14     symbols(symbols),
15     attributes(attributes),
16     character(source.current()) {}
17
18 void Tokenizer::scan() {
19     // Automata state: LOOP
20     while (!source.done()) {
21         switch (attributes.lookup(character)) {
22             // Automata state: WHITESPACE
23             case Attribute::Whitespace:
24                 scanWhitespaces();
25                 break;
26
27             // Automata state: INTEGER
28             case Attribute::Digit:
29                 scanInteger();
30                 break;
31
32             // Automata state: STRING
33             case Attribute::Letter:
34                 scanString();
35                 break;
36
37             // Automata state: COMMENT_START
38             case Attribute::Comment:
39                 scanComment();
40                 break;
41
42             // Automata state: DELIMITER
43             case Attribute::Delimiter:
44                 scanDelimiter();
45                 break;
46
47             // Automata state: ERROR
48             case Attribute::Invalid:
49                 scanInvalid();
50                 break;
51         }
52     }
53     // Automata state: END
54 }
55
56 void Tokenizer::scanWhitespaces() {
57     while (!source.done() &&
58         attributes.lookup(character) == Attribute::Whitespace) {
59         character = source.read();
60     }
61 }
62
```



```

63 void Tokenizer::scanInteger() {
64     while (!source.done() && attributes.lookup(character) == Attribute::Digit) {
65         token += character;
66         character = source.read();
67     };
68
69     // Automata state: WRITE
70     code = symbols.resolveLiteral(token);
71
72     addToken({
73         .code = code,
74         .row = source.row(),
75         .column = source.column() -
76             token.length(), // Point to first character of the token
77     });
78
79     token.clear();
80 }
81
82 void Tokenizer::scanString() {
83     while (!source.done() &&
84         (attributes.lookup(character) == Attribute::Letter ||
85          attributes.lookup(character) == Attribute::Digit)) {
86         token += character;
87         character = source.read();
88     }
89
90     // Automata state: WRITE
91     if (symbols.isKeyword(token)) {
92         code = symbols.resolveKeyword(token);
93     } else {
94         code = symbols.resolveIdentifier(token);
95     }
96
97     addToken({
98         .code = code,
99         .row = source.row(),
100         .column = source.column() -
101             token.length(), // Point to first character of the token
102     });
103
104     token.clear();
105 }
106
107 void Tokenizer::scanComment() {
108     size_t startRow = source.row();
109     size_t startCol = source.column();
110
111     character = source.read();
112
113     // Automata state: WRITE
114     if (character != '*') {
115         addToken({
116             .code = static_cast<size_t>('('),
117             .row = startRow,
118             .column = startCol,
119         });
120
121         return;
122     }
123
124     character = source.read();
125
126     // Automata state: COMMENT
127     while (!source.done()) {
128         // Automata state: COMMENT_CLOSE
129         if (character == '*') {

```

```

130         character = source.read();
131
132         // Automata state: COMMENT_END
133         if (character == ')') {
134             character = source.read();
135
136             return;
137         }
138     } else {
139         character = source.read();
140     }
141 }
142
143 // Automata state: ERROR
144 addError({
145     .message = "Comment not closed",
146     .row = startRow,
147     .column = startCol,
148 });
149 }
150
151 void Tokenizer::scanDelimiter() {
152     addToken({
153         .code = static_cast<size_t>(character),
154         .row = source.row(),
155         .column = source.column(),
156     });
157
158     character = source.read();
159 }
160
161 void Tokenizer::scanInvalid() {
162     addError({
163         .message = std::format("Invalid character '{}'", character),
164         .row = source.row(),
165         .column = source.column(),
166     });
167
168     character = source.read();
169 }
170
171 const std::vector<Token>& Tokenizer::tokens() const {
172     return _tokens;
173 }
174
175 const std::vector<Error>& Tokenizer::errors() const {
176     return _errors;
177 }
178
179 void Tokenizer::addToken(const Token& token) {
180     _tokens.push_back(token);
181 }
182
183 void Tokenizer::addError(const Error& error) {
184     _errors.push_back(error);
185 }

```

src/TokensView/TokensView.hpp

```
1 #pragma once
2
3 #include <iostream>
4 #include <vector>
5
6 #include "SymbolStore.hpp"
7 #include "Tokenizer.hpp"
8
9 class TokensView {
10 private:
11     std::ostream& out;
12
13     static std::string typeLabel(SymbolType type);
14
15 public:
16     TokensView(std::ostream& out);
17
18     void print(
19         const std::vector<Token>& tokens,
20         const SymbolStore& symbols
21     ) const;
22 };
```

src/TokensView/TokensView.cpp

```
1 #include "TokensView.hpp"
2
3 #include <format>
4
5 TokensView::TokensView(std::ostream& out) : out(out) {}
6
7 std::string TokensView::typeLabel(SymbolType type) {
8     switch (type) {
9         case SymbolType::Delimiter:
10             return "Delimiter";
11         case SymbolType::Keyword:
12             return "Keyword";
13         case SymbolType::Literal:
14             return "Literal";
15         case SymbolType::Identifier:
16             return "Identifier";
17     }
18 }
19
20 void TokensView::print(
21     const std::vector<Token>& tokens,
22     const SymbolStore& symbols
23 ) const {
24     std::string border = "+-----+-----+-----+-----+";
25
26     std::string header = std::format(
27         "| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |", "Code", "Row", "Col",
28         "Type", "Value"
29     );
30
31     out << border << "\n";
32     out << header << "\n";
33     out << border << "\n";
34
35     for (const auto& token : tokens) {
36         auto value = symbols.lookup(token.code);
37         auto type = typeLabel(symbols.lookupType(token.code));
38
39         out << std::format(
40             "| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |", token.code,
41             token.row + 1, token.column + 1, type, value
42         )
43         << "\n";
44     }
45
46     out << border << "\n";
47 }
```